ON METACRAFT

The Reimplementation Cost Collapse

Sandra Schipal

draft

On MetaCraft
Sandra Schipal

Draft. Not for sale.

Typeset with DesignCraft. Print profile: mcp-central-docs/standards/BOOK_PRINT_PROFILE.md.

Contents

Preface · 5

I. The Breach · 7

II. The Mechanizers · 11

III. No Moat Equation · 15

IV. The Stack · 19

V. Locating the Present (living, revision-dated) · 23

VI. Thresholds · 31

VII. Dark Megawatts · 37

VIII. The Physical Ladder · 41

IX. MetaCraft · 45

X. The Physical Firewall · 49

Side: The Dark Mirror (governance only) · 51

Preface

This book was started on 2026-10-13, thirteen days after the first commit to a repository called `storytold/photocraft`. By the time the first draft existed, the same organization had published thirteen desktop applications, each aimed at a named incumbent, each released roughly once a day. The book is therefore written from inside the event it describes. Some of its dates will be wrong by the time anyone reads it. Chapter V exists to absorb that: it is a living ledger, revised in place and dated at every revision.

Two levels of authorship

The first draft of the argument came from Gemini, in a chat chain, on 2026-10-13. It is preserved word for word in `docs/GEMINI_DRAFT_2026-10-13.md`. Claude Opus 5.5 reviewed it the same day, checked its claims against public repositories and press, and disagreed with parts of it. Those reviews are kept as dated files (`docs/REVIEW_*`), never edited after their date. Sandra Schipal judges between the models and owns the text.

This arrangement is not decoration. A book about software built by agent swarms, drafted by competing models and judged by one human, is a small instance of its own subject. Where a model's judgment survives into the text, the commit carries a `Co-Authored-By:` trailer.

What this book claims

Proprietary desktop software was defended less by law, and less by genius, than by the labor cost of reimplementing it. In 2026, agent harnesses running on cheap compute collapsed that cost. The law had permitted clean-room reimplementation for decades. What remains of the moat after the collapse is switching cost, distribution, certification, liability, and a narrow, decaying band of patents.

What this book does not do

- It does not give legal advice. Where it discusses doctrine, it cites the case and the date. Ask counsel.

- It does not give investment advice. Chapter VI separates exposed vendors from defended ones; that is analysis, not a trade.
- It does not teach offensive technique. The same harness that clones a photo editor can be pointed at contested domains. The book names those domains once, in a side file, at the level of institutions and governance only (Rule 2, `CONTRIBUTING.md`).

How to read it

Chapters I-V establish what happened and when. Chapter VI lists what still holds and who holds it. Chapter VII is about the compute that pays for the collapse. Chapters VIII and IX follow the method off the screen and up the autonomy ladder. Chapter X sets the physical limits.

Every unverified claim carries a `[construction]` flag. A chapter with no flags left is a finished chapter. At 2026-10-13 there are none.

Fleet link

The sister books (`on-ai-takeover`, `on-hyperwar`, `on-hyperart`) use the same ladder method on the economy, on war and on art. This one is about the tools.

I. The Breach

`storytold` org, PhotoCraft README / AGENTS.md / ROADMAP.md, press 2026-10-07..18, Goliath install check 2026-10-18, `mcp-central-docs/projects/adobe-offensive/01` (2026-10-08).

Thirteen days

On 2026-09-30 a developer named Brandon Thomas made the first commit to a repository called `storytold/photocraft`: an image editor, written in Rust from scratch, described as a clean-room reimplementation of Adobe Photoshop. Four more repositories were created the same day: VectorCraft for Illustrator, FilmCraft for Premiere, LightCraft for Lightroom, PdfCraft for Acrobat. EffectCraft and DesignCraft, for After Effects and InDesign, followed the next day. All seven shipped their first releases on 2026-10-02. The suite went public at getartcraft.com/apps over the weekend of 2026-10-04 `[construction: confirm site launch date]`.

On 2026-10-07 Ars Technica ran the story under the headline “Software is over”. The same day, five more repositories appeared: WordCraft, GridCraft and DeckCraft for Microsoft Word, Excel and PowerPoint; SoundCraft for Avid Pro Tools; CADCraft for Autodesk AutoCAD. FigCraft, for Figma, followed on 2026-10-09.

By 2026-10-12 the thirteen apps had published 93 releases between them, about one release per app per day. PhotoCraft had 55,197 GitHub stars in the 2026-10-13 snapshot. The 2026-10-18 snapshot (`evidence/2026-10-18.json`): 67,027 stars, 12 releases, latest tag v0.10.0; suite-wide 120 releases and 167,586 stars. Inflow cooled after an Oct 7-12 spike. Stars are weather. Chapter V holds the living count. PhotoCraft is dual-licensed MIT or Apache-2.0. The generative studio is a separate product, `storytold/artcraft`, not one of the thirteen.

Its commit log since 2026-09-30 held 1,511 commits and about 1,579 `Co-Authored-By: Claude` lines as of 2026-10-13: nearly every change had an agent as co-author. `[construction: re-count Co-Authored-By at next evidence run]`

Not a lone genius, not a corporation

The Gemini draft described the origin as a single experienced developer. That is half the picture. Thomas leads; press reports describe fifteen years of programming and about three years as chief executive of the AI start-up Storyteller.ai [construction: verify; funding history open]. He has said he worked exclusively with AI coding tools from early 2026 and settled on Claude Opus 5.5 [construction: primary source]. The commit log shows the rest: parallel agent instances writing most of the code, and, within days of launch, a crowd of human contributors. 227 distinct names had committed to PhotoCraft by 2026-10-11.

The correct unit is neither a person nor a company. It is a lead, a method and a swarm.

What was claimed, what was shipped

The founder's claims ran ahead of the software. Early posts spoke of reaching full parity with Adobe within a month; he later revised this to months rather than years [construction: HN thread ids in adobe-offensive/01]. Reviewers found the apps glitchy, incomplete and early. The project itself agrees: PhotoCraft's own roadmap, reviewed 2026-10-11, rates it about 45 percent ready for real work, with feature breadth near 76 percent.

Counting bugs on day thirteen measures the wrong thing. Discovery is being done for free by thousands of users filing issues; fixing is being done in parallel by agents; every fix lands in a shared Rust core used by all thirteen apps. The number to watch is not the bug count but the slope of the parity curve.

The number that matters

The PhotoCraft roadmap contains one line that should open every history of this period. It estimates the remaining distance to beta at 1,100 to 1,800 hours, measured in Opus 5.5 agent wall-clock hours for one agent working sequentially, calibrated from the repository's own 1,004 merged pull requests, and notes that about 80 percent of the work parallelizes.

Adobe's Photoshop organization measures its work in engineers and years [construction: dated headcount source]. PhotoCraft measures its remaining work in model time. That difference is the breach.

What the moat was

The Gemini draft opened with a sentence worth keeping: the moats were never laws of physics. They were the cost of human keystrokes. Chapter III shows the law never protected what the clones copy. Chapter IV shows how the method works. This chapter only fixes the date: between 2026-09-30 and 2026-10-12, the labor cost of reimplementing professional desktop software stopped being the barrier it had been for forty years.

Fleet link

on-ai-takeover: the per-seat subscription line item, seen from the buyer's budget, that this chapter makes optional. on-hyperart: the Crafts are how a generated picture becomes a PSD, AI, or PREL a client can still open. on-hyperwar: dual-use consumer stack versus exquisite incumbent; analogical only, no munitions software.

II. The Mechanizers

revised per Reviews A and C. Gemini and Sandra review pending.

The guild's defense

Every monopoly on making things has defended itself the same way when a machine arrived: the machine's output is inferior. The claim is usually true on the first day and false within a decade. The interesting question is never whether the claim is true but how long it stays true, and what the incumbent does with the time.

The pattern has six dated instances worth knowing.

Lyon, 1804

Joseph Marie Jacquard's loom controlled the lifting of warp threads with a chain of punched cards `[construction: patent and demonstration date, 1801-1804]`. Before it, a figured silk required a drawboy perched on the loom, pulling cords in sequence for hours. The pattern lived in the drawboy's attention. After it, the pattern lived in a stack of cards that could be copied, stored and run again.

The weavers of Lyon resisted; looms were reportedly burned. The resistance failed for a reason that recurs in this book: the cards were cheaper to check than the drawboy. A card either has a hole or it does not. The pattern became data, and data can be verified.

Highland Park, 1913

Ford's moving assembly line did not invent the automobile. It removed the coachbuilder's monopoly on assembling one. Interchangeable parts, made to tolerances a gauge could check, let unskilled workers do in minutes what had taken a craftsman hours `[construction: Model T chassis assembly time before/after, 1913-1914]`. Again the enabling condition was cheap verification: a part fits the gauge or it does not.

Cambridge, Massachusetts, 1983

On 1983-09-27 Richard Stallman announced the GNU project, a free reim-

plementation of Unix. On 1991-08-25 Linus Torvalds announced a kernel to go with it. Together they broke the licensing monopoly on Unix, not by lawsuit but by rewriting it, interface by interface, against a published standard (POSIX) and an observable original.

This is the closest historical precedent to ArtCraft, and it is also a warning. GNU/Linux won the server and lost the desktop. It reimplemented the interface programs used, not the interface people used. Chapter III returns to that difference; it is the hinge of the book.

Toronto and San Diego, 1986

Rumelhart, Hinton and Williams published backpropagation in *Nature* on 1986-10-09. It took until 2012, and a GPU, for the method to beat hand-built feature engineering decisively (AlexNet, ImageNet 2012). The monopoly broken was that of the expert who wrote the rules by hand. The mechanism was statistical learning from data the expert did not curate.

Santa Clara, 2006

NVIDIA's CUDA `[construction: release date, 2006-11 or 2007-02]` turned a graphics card into a general matrix engine. It broke the dominance of scalar general-purpose processors for numerical work. Without it, neither the 2012 result nor the 2026 agent swarms exist.

San Francisco, 2026

The sixth mechanizer has no single name. It is a method: a parity document scored against the installed original, a reference implementation the agents can test against, a budget in agent-hours, and a swarm of model instances writing code in parallel. Chapter I dates its public arrival to 2026-09-30.

The monopoly it breaks is the labor cost of reimplementation. The guild defense has already been mounted: the clones are buggy, incomplete, alpha. All three are true on day thirteen.

A counter-case: Bessemer

Not every mechanizer was robbed. Henry Bessemer patented his converter process in 1855-1856, licensed it, and is estimated to have earned over a mil-

lion pounds in royalties before the patent expired in 1870. When William Kelly's rival American patent appeared, it was bought and pooled with Bessemer's; mills paid under both. The law worked as designed.

Bessemer matters here because he is the exception that shows the rule's real shape: patents protected a physical process with a twenty-year clock. The clones of Chapter I attack software whose core is older than twenty years. Chapter III takes up the difference.

The editorial correction

The Gemini draft listed Dario Amodei as a mechanizer, for constitutional scaffolding of frontier models. That is a brake, not a mechanizer: it constrains a machine; it breaks no guild. It moves to Chapter VI as an attachment point.

Fleet link

on-hyperart Ch II traces the same "inferior imitation" defense in painting, voice and animation. Same defense, same clock.

III. No Moat Equation

(rebuilt), Review C, `mcp-central-docs/projects/adobe-offensive/14-legal-deep-dive.md` (2026-10-11), Sandra on adhesive ToS 2026-10-18. Not legal advice.

The wrong question

The Gemini draft called the incumbents' legal position a bluff: you cannot copyright a Bezier curve. That is true and beside the point. Copyright never protected the Bezier curve, the alpha-compositing formula or the matrix multiply. Nobody believed it did. The law was not the moat.

The right question is why, if reimplementation was legal for forty years, it was so rare.

What the law always allowed

The doctrine is old and consistent.

- **Ideas are free.** 17 U.S.C. s.102(b) excludes any idea, procedure, process, system or method of operation from copyright. *Baker v. Selden*

  (1879) drew the line between a bookkeeping system and the book that explained it.

- **Menus are a method of operation.** *Lotus v. Borland* (1st Cir. 1995, affirmed by an equally divided Supreme Court in 1996) held the Lotus 1-2-3 menu command hierarchy uncopyrightable.
- **Studying the original is allowed.** *Sega v. Accolade* (9th Cir. 1992) and *Sony v. Connectix* (9th Cir. 2000) held intermediate copying for the purpose of understanding unprotected elements to be fair use. Both defendants disassembled code. Watching a program run is milder.
- **Reimplementing an interface can be fair use.** *Google v. Oracle* (US Supreme Court, 2021-04-05) held Google's reuse of Java's declaring code fair use, weighing the investment programmers had made in learning the interface.

- **In Europe, observation is a right the license cannot remove.** Software Directive 2009/24/EC Art. 5(3) lets a licensee observe, study and test a program to determine its underlying ideas; Art. 8 voids contract terms to the contrary. *SAS Institute v. World Programming* (CJEU C-406/10, 2012-05-02): functionality, programming language and data file formats are not protected; a licensee who observed a learning edition to reproduce functionality acted lawfully.

The Phoenix BIOS clean room (1984) is the industrial template: one team studies the original and writes a specification; a lawyer vets it; a second team that never saw the original writes the code. The wall exists to rebut an inference of copying. It is evidence, not a legal requirement.

So why was it rare?

Because it was expensive. A clean-room Photoshop was always lawful and always a decade of skilled labor. GIMP (1996), Krita, Inkscape and LibreOffice show what volunteer labor could reach: capable programs that deliberately did not imitate the incumbents' interfaces. They reimplemented functions. They did not reimplement habits.

That is the moat this book is about. **Switching cost** is the time a trained user loses moving to a program that works differently. For a professional with ten thousand hours in Photoshop, it is the single largest cost of leaving. Microsoft Office users did not stay away from LibreOffice because LibreOffice could not open their files. They stayed because their hands knew where things were.

Google v. Oracle is the doctrine for exactly this cost: the Court weighed the investment users had made in learning an interface as a reason reimplementation was fair. PhotoCraft's own pitch is the same argument told for end users: the menus, shortcuts and panels your hands already know. The legal theory and the product strategy are one sentence.

What the clones change

The ArtCraft method (Chapter IV) attacks switching cost directly. Its parity documents score each app against the installed incumbent, menu item by menu item. At 2026-10-11 PhotoCraft reported every Photoshop menu item present (628 of 628) and 56 of 68 tools. A faithful clone, accurate on the sur-

face and several levels down, makes the trained hand portable.

Agents made that affordable. Nothing in the law changed.

What still binds

Four legal surfaces survive the clean room. They are thresholds, handed to Chapter VI with owners and dates.

1. **Patents.** Independent creation is no defense. The surface is real but narrow and decaying: twenty-year terms, *Alice v. CLS Bank* (2014-06-19) against abstract software claims in the US, and the EPO's technical-character test, which since *T 208/84 Vicom* (1986-07-15) has treated image processing as technical. Live risk clusters in features newer than about 2006, and those are the features the clones lack.
2. **Trade dress.** Lanham Act s.43(a). The most likely first move by an incumbent is a letter about look, not code. The defense is that familiarity is functional; the cheap remedy is to restyle the chrome.
3. **Contract.** End-user license clauses against reverse engineering can bind a US licensee who clicked (*Bowers v. Baystate*, Fed. Cir. 2003). In the EU, Art. 8 voids them as far as observation goes. They are not a moat against clean-room reimplementation. See below.
4. **Format licenses.** Mostly waived by the incumbents themselves to win ISO standardization: Adobe's royalty-free patent license for ISO 32000-1:2008 (PDF 1.7), Microsoft's Open Specification Promise for OOXML (scope disputed by FSFE and SFLC in 2008). PDF 2.0 (ISO 32000-2,

   2017) and Adobe XFA sit outside the PDF license.

Where the AI clean room is weaker than the label

Honesty requires two caveats.

- **The wall.** If one model context both watched the original and wrote the code, there is no wall in the Phoenix sense. Separate observer and implementer contexts with a vetted specification between them would rebuild it. Whether ArtCraft does this is not public. `[construction]`
- **The weights.** The model was trained on public material that includes

tutorials, documentation and screenshots of the incumbent. Clean cannot be proven at the level of weights. The test remains the output: protected expression must be shown in the clone. Fresh Rust against decades-old C++ is unlikely to show it.

As of 2026-10-18 no cease-and-desist, takedown or suit against the Crafting Apps had been reported in press roundups `[construction: re-check github/dmca and dockets at each revision]`.

The ToS that is not a threshold

Adobe's terms, as discussed in October 2026 coverage, include language forbidding monitoring or tracking an application's inputs and outputs "in order to recreate that system." Counsel pieces treat this as a live risk. This book does not.

A term that says "do not look at our product in order to compete" is not a property right. It is a contract of adhesion trying to claw back what copyright already excluded: ideas, procedures, methods of operation (s.102(b)), observation of a running program (Sega, Connectix; Directive Art. 5(3)), and, in Europe, a right the license cannot remove (Art. 8, SAS). Sandra's position, dated 2026-10-18: the clause is arbitrary, based on thin air, a no-hoper as a moat for Adobe or any other vendor. A letter may still be sent. Distribution (Chapter VI, threshold D) may still pinch. The term itself does not convert looking into infringement.

Bowers at most binds the person who clicked that EULA. It does not bind a team that never took the license. It does not survive Art. 8. This chapter therefore does not promote anti-clone ToS language to an attachment point.

Fleet link

on-hyperart Ch VII: copyright filters as a failed control case. There the law tried to protect style and could not; here it never tried. The ToS move is the same wish in contract clothes.

IV. The Stack

PhotoCraft README / AGENTS.md / ROADMAP.md (2026-10-11), Review B. Gemini and Sandra review pending.

The inversion

A desktop application used to be a window with a program behind it. The window was the product; the program existed to serve the mouse. The Crafting Apps invert this. The program is a document model and a list of commands. The window is one of four ways to call those commands, and not the most important one.

```
human intent / agent
|
v
command registry  <---  UI (egui)  |  CLI  |  JSON channel  |  MCP
server
|
v
document model  ->  CPU reference compositor  <== compare ==>  GPU
compositor
|
v
files (PSD, PDF, DOCX, ...)  /  pixels  /  audio  /  drawings
```

Everything is a command

PhotoCraft's contributor rules start with one sentence: everything is a command. Every user-visible behavior is registered once in the engine, with an id, a label, a menu path, a shortcut, documented parameters, an enabled-check and a run function. The UI, the command line, the JSON control channel and the MCP server all dispatch the same ids. At 2026-10-11 the registry held more than 500 commands.

The consequence is easy to miss. Agent control is not a feature added after the fact. An app built this way cannot have a function the agent cannot reach, because there is no other way to add a function. The Gemini draft called MCP "the real nervous system". It is more exact to say there is only one nervous system, and the human hand is one of its four inputs.

Goliath, 2026-10-18: Sandra reports CLI and MCP on the installed Crafts are usable. That is an operator fact about this machine, not a benchmark. Cursor's user `mcp.json` on Goliath does not register the Craft servers; the control plane is the app's own CLI / JSON / MCP, as the READMEs describe.

Native, not wrapped

The apps are written in Rust only. The rules forbid JavaScript, webview shells and Electron. The desktop build draws with egui on wgpu; the web build is the same Rust compiled to WebAssembly. One codebase reaches macOS, Windows, Linux and the browser; PhotoCraft also builds for FreeBSD. FilmCraft alone had no web build at 2026-10-13.

The Gemini draft promised cold starts under twenty milliseconds and memory in megabytes rather than gigabytes. That may be true; it is not yet measured in public. `[construction: measure named app vs incumbent on named hardware; PhotoCraft has a perf/ directory]`

The oracle

This is the part the Gemini draft missed, and it is the part that makes the rest possible.

PhotoCraft composites images twice. A CPU compositor, written for clarity, is the reference. A GPU compositor, written for speed, is tested against it. The PSD reader is tested against files authored in Photoshop itself and kept in a separate, checksum-pinned corpus. Every command is fuzzed with adversarial parameters by a test called `panic_hunt`; a crash on bad input fails the build. A parity floor records how many incumbent menu items are live and may only rise.

An oracle is anything that can say, cheaply and without a human, whether the output is right. Agents can write code quickly. What they cannot do without an oracle is know when they are wrong. Every part of this stack that an agent touches has one.

Rule, stated once here and used for the rest of the book: **agent synthesis scales exactly as far as cheap verification reaches.**

The rules agents follow

The code rules read like a contract written for machines:

- No `unsafe` code anywhere but one isolated crate for pen-tablet input.
- No `unwrap`, `expect` or `panic` outside tests, enforced by the linter.
- No assumption of 8-bit color or sRGB in any public interface.
- No proprietary code, shaders, profiles or assets; implement from public specifications and observation.
- Every change comes with tests; every crash fix with a test that crashed before the fix.

Shared rules across all the Crafting Apps live in a private repository called `craftrules`. Shared Rust libraries live in `craft-libs`, pinned by revision.

The budget

The PhotoCraft roadmap does something no software plan known to this book has done before. It gives the distance to beta, 1,100 to 1,800 hours, in Opus 5.5 agent wall-clock hours for one agent working sequentially, calibrated from 1,004 merged pull requests, and notes that about 80 percent of the work parallelizes.

A project that budgets its future in model time has changed what a software project is. Labor has become a line item priced in tokens. Chapter VII is about who pays for them.

Fleet link

on-ai-takeover: MCP as the interface that turns seat licenses into API calls. The buyer's side of the same stack.

V. Locating the Present (living, revision-dated)

Rule: one row per dated public artifact. No row without evidence. Public catalog: getartcraft.com/apps (“Thirteen apps. One craft.”), accessed 2026-10-18. GitHub org `storytold`. Machine check: hostname Goliath, ArtCraft Launcher v0.5.1.

What the thirteen Crafts are

App	Incumbent workflow	What it is (project’s words, 2026-10-18)	Repo created
PhotoCraft	Photoshop	Image editing: layers, masks, type, real PSD files	2026-09-30
VectorCraft	Illustrator	Vector illustration	2026-09-30
FilmCraft	Premiere	Video editing, color and sound	2026-09-30
LightCraft	Lightroom	Photo library and raw development	2026-09-30
PdfCraft	Acrobat	Reading, organizing and protecting PDFs	2026-09-30
EffectCraft	After Effects	Motion graphics and visual effects	2026-10-01
DesignCraft	InDesign	Page layout and publishing	2026-10-01

WordCraft	Word	Word processing; ribbon, styles, .docx	2026-10-07
GridCraft	Excel	Spreadsheets and data	2026-10-07
Deck-Craft	PowerPoint	Presentations and slide decks	2026-10-07
Sound-Craft	Pro Tools	Audio recording and editing	2026-10-07
CADCraft	AutoCAD	CAD/drafting; command line, snaps, DXF/DWG	2026-10-07
FigCraft	Figma	Interface design and prototyping	2026-10-09

Shared conventions, stated by the project: clean-room, pure Rust, native on macOS/Windows/Linux, WebAssembly except FilmCraft as of 2026-10-13, agent-drivable (CLI, JSON channel, MCP). Dual MIT/Apache-2.0 on PhotoCraft; CADCraft and WordCraft READMEs say the same. Early alpha. PhotoCraft's own 2026-10-18 README: not yet a Photoshop replacement for daily professional work.

Not in the thirteen, and not a Crafting App:

- **ArtCraft** (`storytold/artcraft`, created 2022-03-31): generative image/video studio ("IDE for artists"). Credits and subscriptions live here, not on the clones. Evidence filter in `book.toml` excludes this repo on purpose.
- **ArtCraft Launcher** (`storytold/craft-launcher`): installer/updater. Goliath: v0.5.1, published 2026-10-18T04:01:09Z, binary at `%LOCALAPPDATA%\Programs\ArtCraft Launcher\artcraft-launcher.exe`.

Hand-kept first-release table (2026-10-13, unchanged)

Repo created	App	Incumbent	First release	Evidence
2026-09-30	PhotoCraft	Photoshop	2026-10-02	github.com/storytold/photocraft
2026-09-30	VectorCraft	Illustrator	2026-10-02	github.com/storytold/vectorcraft
2026-09-30	FilmCraft	Premiere	2026-10-02	github.com/storytold/filmcraft
2026-09-30	LightCraft	Lightroom	2026-10-02	github.com/storytold/lightcraft
2026-09-30	PdfCraft	Acrobat	2026-10-02	github.com/storytold/pdfcraft
2026-10-01	EffectCraft	After Effects	2026-10-02	github.com/storytold/effectcraft
2026-10-01	DesignCraft	InDesign	2026-10-02	github.com/storytold/designcraft
2026-10-07	WordCraft	Word	2026-10-07	github.com/storytold/wordcraft
2026-10-07	GridCraft	Excel	2026-10-08	github.com/storytold/gridcraft
2026-10-07	DeckCraft	PowerPoint	2026-10-08	github.com/storytold/deckcraft

2026-10-07	SoundCraft	Pro Tools	2026-10-08	github.com/storytold/soundcraft
2026-10-07	CADCraft	AutoCAD	2026-10-08	github.com/storytold/cadcraft
2026-10-09	FigCraft	Figma	2026-10-10	github.com/storytold/figcraft

Goliath install (dated 2026-10-18)

Hostname `Goliath`. ArtCraft Launcher v0.5.1 installed. Roaming app-data folders present under `%APPDATA%` for PhotoCraft, VectorCraft, FilmCraft, LightCraft, PdfCraft, DesignCraft, WordCraft, GridCraft, DeckCraft, SoundCraft, CADCraft, FigCraft. PhotoCraft log: version 0.9.0 launched 2026-10-15T23:08:56Z. Sandra states CLI and MCP on the installed Crafts are usable on this machine as of 2026-10-18. Cursor's user `mcp.json` on Goliath does not list Craft servers; agent control is per-app (CLI / JSON / MCP), not a Cursor-global registration.

A `%APPDATA%\lookcraft` folder exists (first seen 2026-10-15). No public `storytold/lookcraft` repository found 2026-10-18. Not added to the thirteen. `[construction: identify or ignore]`

EffectCraft has no roaming folder on Goliath at this check. Website still lists it. `[construction: confirm install vs unused]`

Live snapshot (generated)

Regenerated by `just evidence` from the GitHub API. The tables above are the hand-kept record; this one moves with each snapshot.

Snapshot 2026-10-18 (`evidence/2026-10-18.json`).

Repo created	Repo	Target	First release	Releases	Latest	Commits	Stars

2026-09-30	story-told/filmcraft	Premiere	2026-10-02	10	v0.9.0	2015	14722
2026-09-30	story-told/lightcraft	Lightroom	2026-10-02	10	v0.9.1	2913	13590
2026-09-30	story-told/pdfcraft	Acrobat	2026-10-02	10	v0.9.0	1237	15710
2026-09-30	story-told/photocraft	Photoshop	2026-10-02	12	v0.10.0	2564	67027
2026-09-30	story-told/vectorcraft	Illustrator	2026-10-02	15	v0.13.1	4022	9777
2026-10-01	story-told/designcraft	InDesign	2026-10-02	10	v0.9.0	2595	4362

2026-10-01	story-told/effectcraft	After Effects	2026-10-02	12	v0.11.1	2060	7250
2026-10-07	story-told/cad-craft	Auto-CAD	2026-10-08	7	v0.8.0	498	5696
2026-10-07	story-told/deckcraft	Power-Point	2026-10-08	7	v0.8.0	909	3746
2026-10-07	story-told/gridcraft	Excel	2026-10-08	7	v0.8.0	1028	5682
2026-10-07	story-told/soundcraft	Pro Tools	2026-10-08	6	v0.8.0	692	3224
2026-10-07	story-told/wordcraft	Word	2026-10-07	7	v0.8.0	859	14239

2026-10-09	story-told/figcraft	Figma	2026-10-10	7	v0.7.0	1157	2561

Legal weather (silence, dated)

As of 2026-10-18 no public Adobe or Autodesk or Microsoft suit, GitHub DMCA, or cease-and-desist against the Crafting Apps was found in press roundups (Ars 2026-10-07, Al Jazeera 2026-10-14, Declic). Counsel commentary clusters on trade dress, names, and patents, not on copyright of menus. See Chapter III on why an EULA "do not observe I/O to clone" clause is not treated as an attachment point.

Pending rows

- Reddit request for an AutoCAD clone: `[construction: thread URL + date; establish whether it preceded or followed 2026-10-07]` (Review D s.5).
- LookCraft roaming folder on Goliath, no public repo.
- EffectCraft install on Goliath.

Columns to add

Agent co-author share, parity % (from each repo's ROADMAP.md), incumbent price per seat per year at the same date.

VI. Thresholds

Sandra on ToS 2026-10-18. Not legal or investment advice.

Method

The sister books describe each surviving control as an attachment point: a place where a named party can still apply a brake, with a date for when it last held. This chapter lists the attachment points that survive the collapse of re-implementation cost. It is meant to be photocopied.

The table

#	Threshold	Who holds it	How it binds	State 2026-10-18
A	Patents	incumbents; USPTO, EPO, courts	method claims; independent creation is no defense	partial and decaying: 20-year terms; post-Alice; live claims cluster in post-2006 features the clones lack
B	Trade dress and trade-mark	incumbents; courts	look of the product; names and marks	real, cheapest first letter: restyle chrome, keep behavior. PhotoCraft's pitch is familiarity; that is the trade-dress surface
C	Format control	spec owners	undocumented or changing formats	weak where specs are public (PSD, OOXML, PDF 1.7); stronger for DWG history and project formats (AEP, PRPROJ)
D	Distribu-tion	Apple, Micro-soft, Google stores; code-signing author-ities; GitHub	notarization, signing, store policy, take-down	strong: the realistic enforcement choke-point. Goliath current-ly runs the suite from the project's launcher, not a store

E	Certification and liability	regulators, insurers, auditors	software that must be certified or audited (medical, avionics, accounting, tax, stamped CAD)	strongest: the process, not the output, is audited. CADCraft is the Craft that hits this first
F	Enterprise bundle	CIOs; the incumbent's cloud	identity, mail, compliance, tenant data, support contracts, indemnity	strong: this is what enterprise software actually sells
G	Networks	banks, accountants, payroll and tax systems	the app is a node in a network the clone does not join	strong for QuickBooks-class targets
H	Model-provider policy	labs	usage policies; constitutions	variable; outside the clone's control

Anti-clone EULA language ("do not observe I/O to recreate") is **not a row**. Chapter III: it is a wish, not a property right. It does not get a letter in this table.

Row H is where the Gemini draft's Amodei line belongs: a brake on the mechanism, held by the lab, not a mechanizer.

Reading the table

CADCraft is the liability exhibit. An ugly PSD is an ugly PSD. A DWG that an agent stamped into a permit packet is a named professional's problem. Threshold E is not "certification" as a slogan. It is: who answers when the drawing is wrong.

Thresholds A to C are the old moats. They were never strong; labor cost hid how weak they were. The clones expose them. Contract terms that forbid looking are not in A-C. They are noise around D.

Thresholds D to G are not about the program. They are about what surrounds it: who may ship it, who must certify it, which network it joins, which

bundle it sits in. They are where the incumbents will move, the way the music industry moved after 2001 from selling copies to licensing catalogs through a service. The law stopped Napster. It did not stop file sharing. The industry survived by finding a moat that still worked.

Who is exposed

The useful split is not by size or sector but by where the moat sits.

- **Exposed:** vendors whose moat is the application itself: its interface, its features, its file format. Desktop creative tools, standalone office applications, single-user design and audio tools.
- **Defended:** vendors whose moat is a network, a bundle, data, a certification or a liability. Enterprise suites with identity and compliance attached; accounting with bank feeds and tax filing; certified engineering tools.

Most large vendors are both. Their exposed products are the ones they sell to individuals and small firms; their defended products are the ones they sell to enterprises. Expect defection from the bottom first.

Where the last brake sits

Threshold D deserves one more sentence because it survives every later chapter. Whoever signs the binary is accountable for it. Swarms can write code; a code-signing certificate is issued to a named party. When Chapter IX asks where the human remains in an autonomous software foundry, the first answer is here.

Fleet link

on-hyperart Ch VI and on-hyperwar Ch VI use the same table form. The thresholds differ; the method is shared.

VII. Dark Megawatts

revised per Review A. Gemini and Sandra review pending.

Who pays for the agent-hours

Chapter IV ended with a budget in agent-hours. Somebody pays for them. The Gemini draft's answer is the most interesting economic claim in its text: the datacenter buildout of the mid-2020s, whatever it does to its investors, will leave behind cheap compute the way the telecom crash left behind cheap fiber.

The claim is half right. This chapter keeps the right half and dates the wrong one.

The fiber precedent

Between 1996 and 2001, carriers laid far more fiber than the traffic of the day required. When the bubble burst, much of it went unlit `[construction: share of US long-haul fiber unlit ca. 2002, dated source]`. Bankrupt carriers sold their networks for a fraction of what they cost. The buyers lit the fiber cheaply over the following decade, and the web of video, cloud and mobile data ran on capacity that someone else had already written off.

The mechanism is general: capital that is destroyed on the balance sheet is not destroyed in the world. A written-off asset that still works has near-zero capital cost to its next owner.

The right half

A GPU written down to zero on an accountant's schedule still computes. Depreciation is an accounting convention; it is not a physical clock. When a newer generation arrives, older racks that are fully paid for do not vanish. Their owners can sell their time cheaply, and for work that does not need the lowest latency, such as batch code synthesis, test runs and fuzzing, they are good enough.

That is precisely the workload of Chapter IV. The agent swarms that build the Crafting Apps do not need to answer a user in two hundred milliseconds. They need many hours of patient compute, and they can wait for it overnight.

The wrong half

The Gemini draft claimed silicon does not rot. Two facts disagree.

1. **Failure at scale.** Meta's Llama 3 report (2024-07) recorded hundreds of unexpected interruptions during a 54-day training run on about 16,000 H100 GPUs, a majority attributed to GPU and memory faults `[construction: verify exact figures]`. Large GPU fleets fail continuously. Old fleets fail more.
2. **Electricity is not amortized.** A fully written-off accelerator still pays full price for every joule. When a newer chip produces several times more tokens per joule, the old rack loses on running cost alone, long before it breaks. Fiber was different: the glass got more valuable when better lasers were fitted at each end. A GPU cannot be upgraded at the ends. Its useful life is set by the efficiency of its successors.

So the dark-fiber analogy holds for years, not decades. `[construction: tokens per joule by generation, A100 / H100 / current, at a stated power price]`

What reaches the clone-builders

The subsidy only matters if it reaches the people running the swarms. That depends on who owns the stranded racks (hyperscalers, specialist cloud providers, lenders who repossess them) and whether spot prices for older hardware actually fall. The evidence to collect is a dated series of spot prices for previous-generation accelerators and of API token prices for the models used to write code `[construction]`.

There is a second subsidy the Gemini draft did not name: the model providers' own price competition. When frontier labs cut token prices to win developers, the cost of an agent-hour falls whether or not any datacenter goes dark.

The honest word

The Gemini draft's subtitle promised a zero-cost frontier. The PhotoCraft roadmap refutes it: 1,100 to 1,800 agent-hours to beta is not zero. The correct claim is that the **marginal cost of replication** is near zero and the **cost of synthesis** is falling fast. The chart this chapter needs is agent-hours to parity multiplied by the price of an hour, plotted over time. That curve, not the word zero, is the argument.

Fleet link

on-ai-takeover: the capital cycle from the investor's side. on-hyperwar: compute as a strategic stockpile.

VIII. The Physical Ladder

(Level 2), restructured per Review E. The ordering by test cost is Opus opinion; see Review E questions 1 and 3. Gemini and Sandra review pending.

Software that makes things

Most professional software outside the office does not produce documents. It produces instructions for a machine: where a needle goes, which needle on a knitting bed lifts, where a blade cuts, how a tool head moves. The design program sits between a person and a machine, and it is often the most expensive and most locked part of the chain.

That makes it the next target. The method of Chapter IV does not care whether the output is a PSD file or a stitch file. It needs a spec, an original to compare against, and a cheap way to check the result.

The ordering rule

Chapter IV stated the rule: agent synthesis scales as far as cheap verification reaches. On a screen, verification is a pixel comparison. Off the screen, it is a physical test, and physical tests have prices. The ladder is ordered by them.

Rung	Domain	Physical check	Cost per check
1	Machine embroidery, home cutting	stitch-out or cut on scrap	cents, minutes, at a kitchen table
2	Machine knitting	a swatch	small
3	Garment pattern and fit	bodies vary, cloth drapes	medium; needs people
4	Agriculture	a growing season	a year
5	Chips	a tape-out	millions

Rung 1: needle and blade

Machine embroidery runs on stitch files (DST, PES, EXP and others) produced by digitizing software: Wilcom, Hatch, Brother PE-Design and others `[construction: current prices and licensing models]`. Home cutting machines run on design apps such as Cricut Design Space, which ties the software to the machine and to a subscription.

The formats are largely documented or long since reverse-engineered by open-source projects `[construction: name and date the main open libraries]`. The hard knowledge is physical: how much a stitch pulls the fabric, how density changes on stretch materials, how underlay holds a satin column. That knowledge is learned by stitching out samples. A sample costs a scrap of cloth and a few minutes. An agent with a camera and a machine can run that loop all night.

Rung 1 is therefore not a distant future. It is plausibly the next step after office software.

Rung 2: the knitting bed

Industrial knitting machines (Shima Seiki, Stoll) are programmed through their makers' design systems `[construction: SDS-ONE APEX, M1plus, licensing]`. A program is a grid of needle actions. A swatch checks it. The loop is slower than embroidery and the machines are expensive, but the check is still small and local.

Rung 3: the garment

Pattern-making software (Gerber AccuMark, Lectra Modaris, Optitex) and 3D garment simulation (CLO3D, Browzwear) turn bodies and cloth into flat pieces. Here the check gets expensive: fit depends on bodies, and bodies vary. The simulation is an oracle only as far as it is trusted, and fashion houses still sew samples. Rung 3 falls more slowly, and falls first where the body is standardized: uniforms, workwear, sizes from published tables.

Rung 4: the field

The Gemini draft's SoilCraft (hydrology and soil models compiled to guidance tracks and irrigation schedules) meets a check measured in seasons. Worse for

the clone, the moat in agriculture is not the planning software. It is the machine's firmware and the right to repair it. That fight is being fought in regulators and courts, not in code `[construction: FTC action against Deere, 2025, status]`.

Rung 5: the chip

Chip-design tools are the most expensive case, covered in Review D. The check is a foundry run that costs millions and the tools only count once a foundry certifies them. Lawyers do not protect this rung. Physics and the foundry do.

Where the moat goes

When the design software is free, lock-in moves to the machine maker: proprietary formats, cloud-required apps, encrypted firmware, warranty terms. Agriculture shows the pattern first. Expect embroidery and cutting machines next, and expect right-to-repair arguments to follow them from the farm to the sewing room.

The hand again

There is a cultural inversion in this chapter that the Gemini draft did not see. For thirty years, the expensive part of a craft hobby or a small textile business was often the software license. When the license is free, the scarce part is the human: the eye for cloth, the hand on the hoop, the judgment about drape. The tools return to the edge, and the skill becomes the thing worth having again.

Fleet link

on-hyperart: folk-art demotion under aesthetic abundance. Here the same abundance runs the other way: it frees the hand rather than replacing it.

IX. MetaCraft

restructured per Review E. The autonomy levels and their reading are Opus opinion; see Review E questions 2-4. Gemini and Sandra review pending.

The craft that makes crafts

Thirteen applications in two weeks is not thirteen projects. It is one method run thirteen times. This chapter names the method as an object of its own: MetaCraft, the foundry that makes Crafts under agent control.

The foundry already exists

Strip PhotoCraft's documents of their subject and what remains is a recipe:

Foundry part	ArtCraft instance (2026-10-11..13)
Specification	parity documents scored against the incumbent; a generated checklist of every incumbent menu item
Ground truth	the incumbent itself, installed and measured (Photoshop 27.11.0, 2026-10-10)
Budget	remaining work in agent wall-clock hours, calibrated from merged pull requests
Checkers	reference implementations, corpora of files made by the incumbent, fuzzing of every command
Standards	shared rules across all apps (craftrules); shared libraries pinned by revision
Output	about one release per app per day

A human chooses the target and approves the rules. Agents do most of the rest. The foundry is not a forecast. It is the commit log.

Five levels

The levels below are a way of dating progress. They are a proposal, not an industry standard.

Level	Who does what	Evidence at 2026-10-13
L0	Humans write the code	the old world

L1	Agents write code against specs and checkers made by humans	ArtCraft now
L2	Agents write the specification by examining the incumbent, and choose targets from demand	partial: the parity checklist is generated
L3	Agents build the checkers themselves	not yet evidenced in public
L4	The foundry improves its own method, rules and pipeline	not yet evidenced in public

Why L3 matters most

Chapters IV and VIII made one claim twice: agent synthesis scales as far as cheap verification reaches, and the order in which incumbents fall is set by the cost of checking a clone. Today, humans build the expensive checkers: the corpus of files made in the real application, the reference implementation, the physical sample.

A foundry whose agents build those checkers themselves removes the constraint that sets the order. Below L3, the storm moves through the industry at the pace humans can build oracles. Above it, at the pace of compute. Watching for the first public evidence of L3 is the single most useful thing a reader of this book can do. `[construction: define what counts as evidence, e.g. a corpus or reference implementation committed with no human author]`

The test bench paradox

A swarm that writes both the product and the tests can agree with its own mistakes. Independence needs a reference outside the swarm. In the cloning phase that reference is the incumbent: installed, measured, driven through the same actions as the clone, its outputs compared.

So the foundry depends on the thing it replaces. The incumbent becomes the clone's test bench. MetaCraft is strongest where an original exists and weakest where it must invent. What happens when the clone surpasses the

original, and must become its own reference, is open. `[construction: Review E question 4]`

Where the humans remain

The sister books ask of every system where a named party can still apply a brake. In the foundry, the attachment points are:

1. **Target choice.** Who decides what to clone next.
2. **Rule approval.** Who signs off the foundry's standards.
3. **Release signing.** Who holds the code-signing certificate (Ch VI, threshold D).
4. **Accountability.** Who answers a court or a regulator.

As agents gain independence, these do not disappear. They concentrate into a few people and a few keys. A responsible history of MetaCraft names them and dates them, rather than describing the removal of the human from the loop. Removal is a claim about the code. Accountability is a fact about the law, and it has not changed.

The application does not die, yet

The Gemini draft predicted that the application would dissolve into ephemeral software, generated on demand and discarded. The evidence of 2026 runs the other way. The foundry produces stable, versioned clones of named products, released daily, tracked against a parity floor that only rises. That is more application, not less.

Disposable software may come later, for problems an automated check can verify completely. Where a process must be certified or audited (Ch VI, threshold E), the stable application survives because the audit needs something stable to audit. This book treats ephemeral software as a hypothesis for a later revision of this chapter.

Fleet link

on-ai-takeover: the same autonomy levels applied to firms rather than applications. A company run as a foundry is the next chapter of that book.

X. The Physical Firewall

revised. Gemini and Sandra review pending.

What does not collapse

This book has described a collapse in the cost of making software. It closes by naming what the collapse does not touch, because a thesis that claims everything claims nothing.

Matter costs energy

Software copies at near-zero marginal cost. Steel, propellant, a five-axis milling machine and a chip fabrication line do not. Each needs mass, energy, capital and time that no prompt supplies. The Gemini draft put it well: you cannot prompt-engineer five tons of liquid oxygen.

The physical ladder of Chapter VIII shows the boundary in practice. The foundry advances quickly where the physical check is cheap (a stitch-out) and slowly where it is expensive (a growing season, a tape-out). The firewall is not a wall. It is a price, and the price rises with mass and energy.

The nanotechnology argument

The futurist tradition imagined software escaping into matter through molecular machines. The Gemini draft dismisses this, citing thermal noise, surface forces and heat. The argument has a dated public form: the 2003 exchange between K. Eric Drexler and Richard Smalley in *Chemical & Engineering News* `[construction: issue date]`. This book does not need to settle it. It needs only to note that in 2026 no software foundry produces matter, and that its effects on the physical world run through machines that already exist and are already owned.

Biology is slow

Designs made in software for living systems still need physical synthesis, reagents and growth, and they move at the speed of cells and of the institutions that oversee laboratories `[construction: cite one dated governance instrument, e.g. synthesis screening guidance]`. The collapse of software labor

cost does not collapse incubation time.

What did collapse

What collapsed is narrower and, for daily life, larger: artificial monopolies over information, coordination and tools. A trained user's habits, once locked to one vendor's program, became portable. A design program that cost more than the machine it drove became free. A software plan became a budget in model time.

The Gemini draft ended on three sentences. Two survive review. The moats are going, not gone; Chapter VI lists the ones that hold. The tools are returning to the edge: to the individual, the small firm, the crafter at a kitchen table. And the code belongs to whoever signs it, which is the attachment point every later revision of this book should watch.

The book in the corpus

This book was drafted by two models and judged by one person, in public repositories, within a fortnight of its subject. It will be read, if it is read at all, partly by models. Every claim is dated and every guess is flagged so that a later reader, human or not, can tell which is which. That is the only defense a book this close to its subject has.

Fleet link

on-hyperart Ch II: the futurists expected the physical singularity first; the informational one arrived instead. This chapter is that thesis applied to tools.

Side: The Dark Mirror (governance only)

Level 3. Governed by Rule 2 (`CONTRIBUTING.md`): institutional facts and governance criteria only. No mechanisms, now or in later revisions.

Why this file exists

The foundry is indifferent to its target. A method that clones a photo editor can be pointed at contested domains. A book that ignored this would be naive; a book that described how would be dangerous. This file does neither. It names the domains, the institutions that govern them, and the attachment points that apply.

Three contested domains

Domain	Main governing instruments	Attachment points
Offensive cyber capability	Wassenaar Arrangement controls on intrusion software (added 2013) [construction]; national computer-misuse law; model-provider usage policies	model providers, compute providers, code hosts
Weapons autonomy	US DoD Directive 3000.09 (updated 2023-01-25); UN CCW Group of Governmental Experts on lethal autonomous weapons; export controls	states, manufacturers, export authorities
Illicit commerce	anti-money-laundering and know-your-customer regimes (FATF standards); controlled-substance and precursor law	financial intermediaries, logistics carriers

Weapons autonomy is the subject of the sister book `on-hyperwar`, which owns that domain under its own hazard rule. This book defers to it.

The pattern

In each domain the binding constraint is the same as in Chapter VI: not the code, but who will run it, host it, pay for it or carry its outputs. The attach-

ment points in this table are institutional: model providers who refuse, compute providers who verify customers, banks and carriers who report. The cost collapse of Chapter I lowers the price of writing software. It does not lower the price of passing through these points, and the book's position is that they are where governance effort belongs.

What this file will never contain

Techniques, architectures, component lists, target selection or any detail that would help an operator rather than a principal. A proposed edit that adds any of these is cut on sight.